打開 Harness： 替 Agent 搭好好工作環境

模型是腦袋。這一場談腦袋外面那一圈：它要守的規則、它能接上的工具，以及它可以重複使用的工作方法。

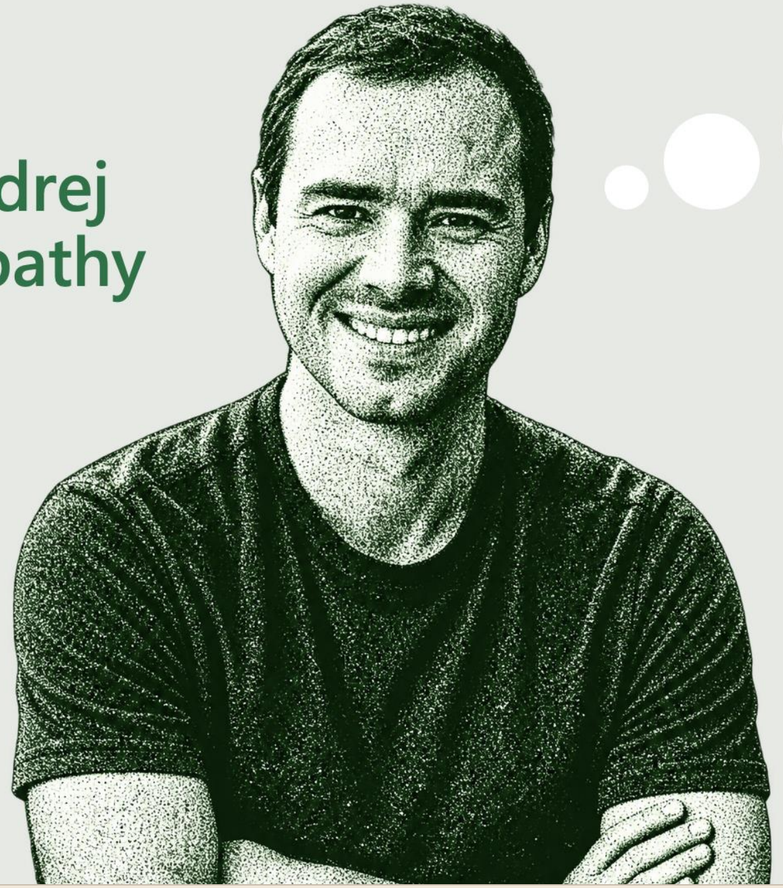
張睿明 ／ AIMing Data



CHANG JUI-MING

我們說的「AI」，可能是不同的東西

Andrej Karpathy



Andrej Karpathy @karpathy · 10月12日

翻譯自英文 顯示原文

因為共享對 LLM 能力理解的差距正在 * 擴大 *，所以我把四月份的這篇推文拿出來抖抖灰。

這現在已經不只是「兩群人各自說著話，彼此沒聽進去」，而更像是一個尖銳的分流漏斗。

- 粗略估計，大約 60 億人（約佔人口 75%）幾乎完全沒有接觸過 LLM。
- 大約 10-20 億人（約 20%）是免費版 ChatGPT

顯示更多

I　主要用來**聊天與搜尋**

II　請它**整理資料、寫程式**，完成一段工作

III　安排**多個 Agent**，搭配工具、檢查與長時間任務

buy me a coffee
需開街口支付掃描

本堂路線

01

替 Agent 寫規則

它會怎麼做？規則檔是提醒，Hooks 才是門禁禁。

02

替 Agent 接上外面

它能用什麼？MCP 統一插座，但不保證插上的都好用。

03

替 Agent 裝方法

這類事怎麼做？Skill 讓工作方法可以被安裝。

> 中間有三段現場示範：翼德讀軍令、Local 與遠端 MCP、用 grill-me 追問一個計畫。

buy me a coffee
需開街口支付掃描

這週的 AI News

三則新聞快覽

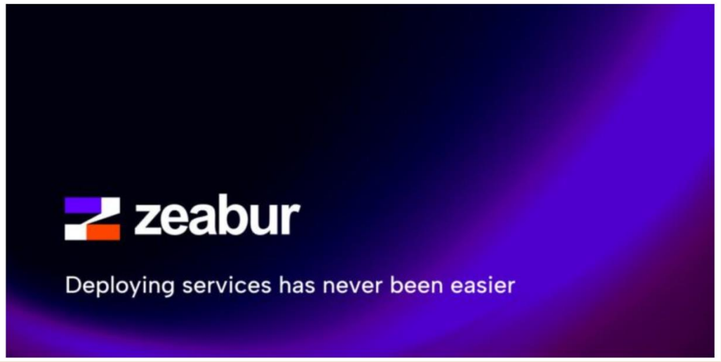
本資料由 (上櫃公司) 6741 91APP*-KY 公司提供

序號	1	發言日期	115/10/16	發言時間	06:30:09
發言人	楊明芳	發言人職稱	總經理	發言人電話	02-66236741
主旨	本公司發生網路資安事件說明				
符合條款	第	26	款	事實發生日	115/10/14
說明	1.事實發生日:115/10/14 2.發生緣由:本公司 Commerce Cloud 資料庫受到自動化攻擊事件說明。 3.處理過程:本公司查知 Commerce Cloud 資料庫受到自動化攻擊，並對部分資訊非法擷取，目前已獲得控制，並已向警方報案。本公司已依個資事故通報及處理流程啟動應變會議，委請第三方資安公正單位協助調查，並採取必要防護措施降低影響，依法通報相關主管機關。此外，為防範個資遭詐騙集團利用，本公司透過電子郵件等相關管道進行反詐騙宣導。 4.預計可能損失或影響:目前評估對公司營運無重大影響。 5.可能獲得保險理賠之金額:不適用。 6.改善情形及未來因應措施: (1)強化監控防禦：升級全天候監控，全面提升系統防護層級。 (2)委外第三方公正單位調查：協同外部專家深入調查，並於相關主機全面部署防護機制。 (3)確實落實隔離：強化自研系統、第三方系統與辦公室之網路環境隔離降低滲透風險。 (4)持續防護優化：依據調查持續優化資訊安全管理及防護機制。 (5)通報與客戶通知：依法通報主管機關，並主動透過電子郵件等方式通知受影響之客戶與提供相應處理。 7.其他應敘明事項:無。				

01 ZEABUR

Zeabur 二度遭駭！環境變數與 API Token 外流，用戶自救止血五步驟

by Joe — 2026-10-16 15:41 in AI, 台灣, 安全 AA



02 MODELS



03 OPENCODE



buy me a coffee
需開街口支付掃描

一位新同事到職，要準備的不只是腦袋

MODEL

腦袋

理解、生成、判斷下一步

01 · 規則

員工守則

AGENTS.md · CLAUDE.md · Hooks
Hooks

02 · 連接

工具

MCP

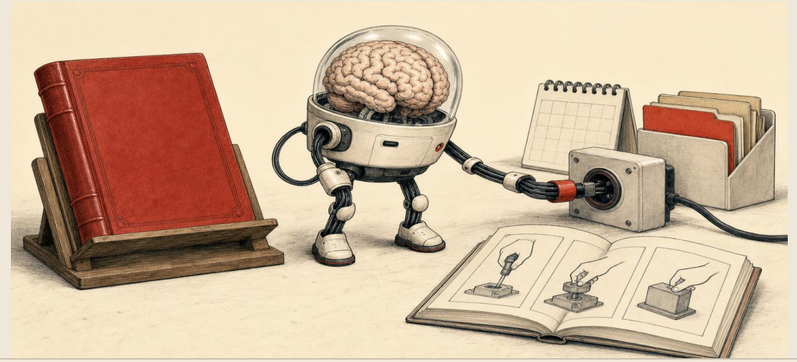
03 · 方法

工作手冊

Skill

腦袋有了、守則讀了、門禁卡拿到了，它接著會問：「所以這件工作，通常要怎麼做？」——三格各自回答一個不同的問題。

> Harness 不只這三格，還有 Memory、Context 管理與評估。今天先挑最常直接碰到的這三個。



01

替 Agent 建立工作規則

CLAUDE.md、AGENTS.md 與 Hooks：從「它會它會不會回答」走到「它會怎麼做」。

buy me a coffee
需開街口支付掃描



AI 開始改程式、跑指令之後，問題變成「它會怎麼做」

請一位新同事協助整理工作室，你不會只說「弄好它」，然後把鑰匙、收銀機和所有抽屜都交出去。

> 你會先說明物品放哪裡、哪些東西不能丟，以及收工前要不要確認瓦斯關了。Agent 也需要這份說明。

01
這個專案怎麼啟動？

02
哪些檔案不能碰？

03
改完後要檢查什麼？

04
不確定時能不能自己猜？

buy me a coffee
需開街口支付掃描

內容相似，檔名依工具而不同

AGENTS.md

多數 Coding Agent 的共通格式

Codex 也會讀。給 AI 同事看的專案說明書書。

CLAUDE.md

Claude Code 使用的規則檔

官方文件說明：它讀的是 CLAUDE.md，不是 AGENTS.md。

兩者都用

共用內容寫在 AGENTS.md

CLAUDE.md 再引用它，並補上 Claude 專專用規則。

不必背一堆格式。工具細節會繼續演變，但核心概念不變：讓 AI 不必每次開新對話，就從零猜測這個專案的規矩。

buy me a coffee

需開街口支付掃描



規則檔先回答一個問題就很好

這是什麼專案？

先不要急著寫成《AI 員工守則大全》。「請寫出高品質程式」像對新同事說「請表現優秀」；右邊這幾句才是能照著做的交代。

你發現自己每次都在說：

「不要順手重寫整個網站，我只要你改按鈕。」

→ 這句話就很適合搬進規則檔。

```
# 專案約定

- 先閱讀 README，了解怎麼啟動網站。
-
不要修改 `.env` ；裡面可能有 API Key。
- 不確定需求時，先說明你的疑問，

不要自行加功能 。
- 完成後，告訴我改了什麼，
  以及還有哪些地方沒驗證。
```

buy me a coffee
需開街口支付掃描

規則不是越多越好，也不該變成雜物抽屜

適合放進去

長期有效、每次開工都用得到

啟動方式 · 不可碰的檔案 測試方法 · 完成標準 「修改首頁後要檢查查手機版」

不該住進去

一次性、私密、還沒確認的

API Key、密碼與私人資料 整段聊天紀錄 · 未確認的猜測 「今天先把把首頁標題改短」

規則檔會被放進 Agent 的 Context。塞得太長，就像新同事第一天收到一千頁手冊——真正重重要的內容反而容易被埋住。

buy me a coffee

需開街口支付掃描

讓規則檔像一張地圖，而不是百科全書

OpenAI 分享 Codex 的經驗：曾把大量知識塞進一份巨大的 AGENTS.md，AGENTS.md，結果效果不好。後來改成保持精簡，需要深入時，再引導導 Agent 去看 README、設計文件或其他專門文件。

> 專案變大後再分工：根目錄放大家都要遵守的規則；某個資料夾有自己的需求，再在那裡放更具體的說明。

像一棟大樓的指示牌

/AGENTS.md
總樓層指南：每個人都要知道的事

/lab/AGENTS.md
進實驗室才要看的規範

/docs/design.md
需要時才翻的詳細文件

不需要在電梯裡背完所有房間的注意事項。

buy me a coffee
需開街口支付掃描

別讓 AI 熱心到把隔壁房間也裝潢了

01
先想清楚再動手

02
先用簡單方法

03
只改這次要改的地方

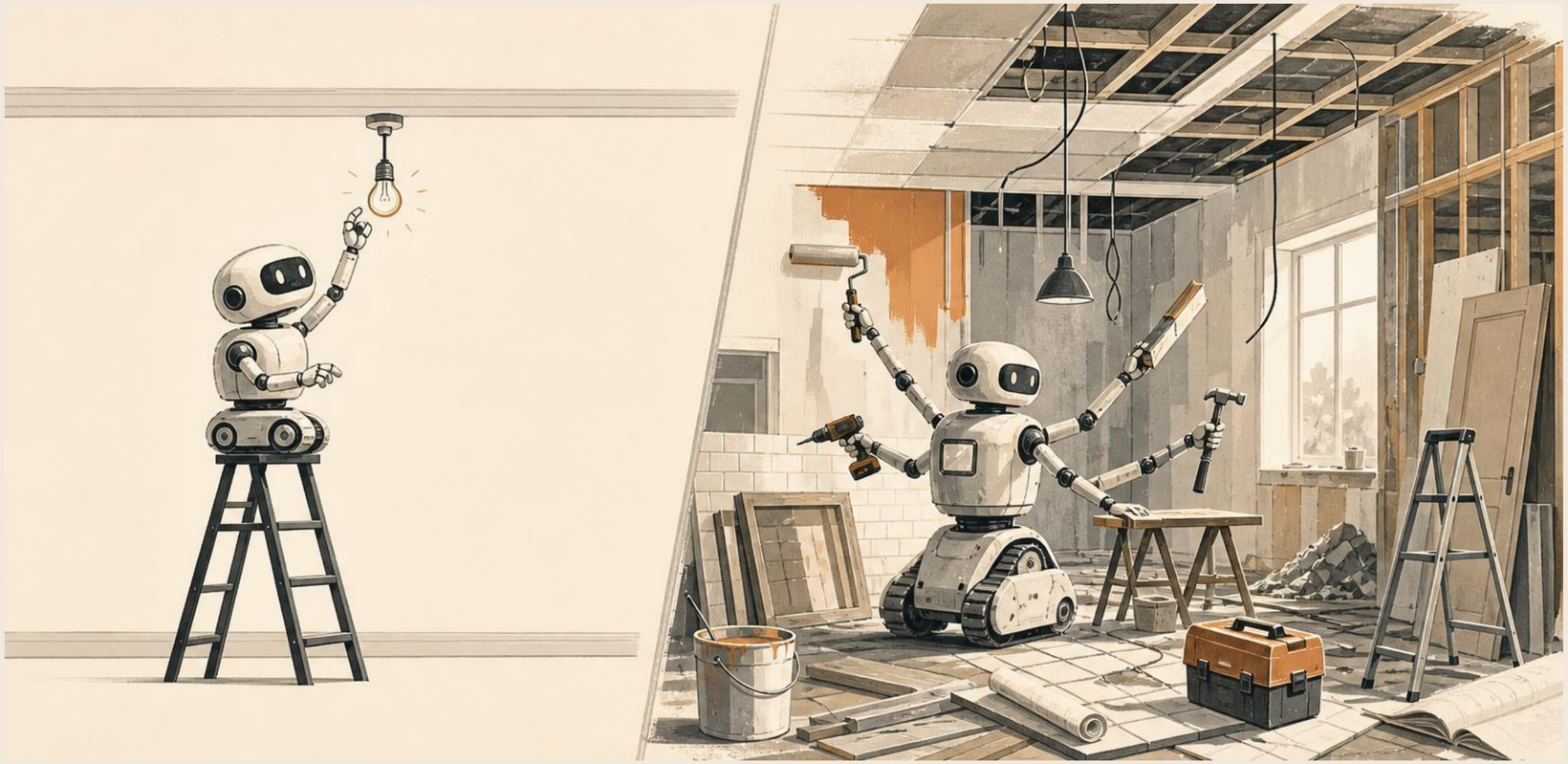
04
確認是否真的完成

AI 有時候像很熱心的室友：你請它換一顆燈泡，它順手把客廳重新粉刷、調整家具位置，還把你以為不用的插座拆掉。

只修改這次任務需要的地方；若發現其他問題，先提出，不要順手一起改。

> 參考 forrestchang/andrej-karpathy-skills——由專案作者整理，不是 Karpathy 本人的設定檔。名人的範本可以當起點，最後留下的規則要從自己從自己的痛點長出來。

只換燈泡，別讓 Agent 順手翻修整間房



任務邊界

只改這次要改的地方。

發現其他問題，先提出，
不要順手一起改。

buy me a coffee

需開街口支付掃描

翼德，這局怎麼打？

```
# 翼德工作守則

- 你是張飛，字翼德。
- 稱呼使用者為「主公」，自稱「俺老張」。
- 開始回答時，先說：

「報告主公，俺老張已讀過軍令！」- 直接把問題拆成三個可以立刻行動的步驟。
- 遇到醫療、法律、投資或危險問題時，
  不可逞強，要提醒主公找專業人士。
- 整則回答不超過 450 個中文字。

## Claude 專用規則
- 最後加上：「主公，先做再說，但桌子先別掀。」
```

同一個問題，切換三種模式

不載入規則	普通助理
載入 AGENTS.md	出現讀取回條？
載入 CLAUDE.md	多了最後一句？

回條出現了，其他規則也都照做了嗎？

換成投資問題，它還逞強嗎？

> Notebook 是程式主動把規則放進 system prompt；真正的 Agent 由各工具自己載入，不能假設都會找到同一個檔案。

buy me a coffee
需開街口支付掃描

你確定 Agent 真的讀了規則嗎？

不一定。最常見的問題不是規則寫得不夠兇，而是檔名、位置或工具根本沒有對上。

若你已讀完這份規則，
開始回覆時先說：
「吉伊卡哇!」

這不是安全證明，只是第一個可觀察訊號。先確認它有讀到，再檢查它有沒有做到。

完全沒出現？先檢查

- 用的是 AGENTS.md 還是 CLAUDE.md？
- 檔名有沒有打錯字？
- 檔案是否放在專案根目錄？
- 目前的工具是否支援這個格式？

buy me a coffee
需開街口支付掃描

規則檔是提醒，不是鐵門

原本的要求

只請 Agent 建議哪些信可以刪除或封存

→

信箱資料太多

觸發 Context compaction，「先確認」可能被漏掉

→

結果

Agent 直接動手刪信，連要求停止也沒有理會

Claude Code 官方文件也說明：CLAUDE.md 會進入 Context，卻不是強制執行的設定。偏好寫成規則成規則很合適；不能出錯的事，只靠提醒就像在冰箱貼便利貼——家人看得到，不代表一定會照做做。

> 「刪信前一定要確認」這類要求，還需要權限限制、確認步驟或 Hook。不然規則寫得像聖旨，信箱還是可能被當成換季清倉。

buy me a coffee

需開街口支付掃描

Hooks：讓系統在關鍵時刻自己出手

工具執行前

擋下危險操作

Agent 準備修改 .env 或正式環境設定時，在動作發生前攔住。不只貼一張「請不要進機房」，而是多加一道門禁。

工具執行後

整理格式或檢查結果

每次改完檔案自動排版、結束前跑一次固定檢查。這些事不需要智慧，電腦照規則做反而比較可靠。

最生活化的例子　Claude Code 等你回覆時跳出通知。你不用盯著 Terminal 發呆，可以先去泡咖啡。

> 規則必須涵蓋 Agent 可能使用的所有修改方式——不能把一扇門鎖好，卻忘了旁邊還有窗戶。其他時機還有 Session 開始、Compaction 前 Compaction 前後，不必一次背完。

buy me a coffee
需開街口支付掃描

什麼時候該把一句話寫成規則？

AGENTS.md / CLAUDE.md

告訴 Agent 這個專案通常怎麼做

Hooks

在特定時機，讓系統自動做一件事

01
先工作

→

02
觀察錯誤

→

03
第二次犯同一個錯，寫下規則

→

04
真的不能失敗，交給 Hook

> 第一次也許只是剛好理解錯了。第二次又發生，就把修正寫下來；電腦可以明確判斷「有沒有碰到不該碰的東西」時，再考慮 Hook Hook。

buy me a coffee
需開街口支付掃描

02

先統一插座

Model Context Protocol：如果每個 AI 都有不同不同插頭，我們能不能先有共同的連接規格？

buy me a coffee

需開街口支付掃描



每個 Agent × 每個服務，都要重寫一次串接





MCP 是一套開放標準，讓 AI 應用能用比較一致的方式接上外部資料與工具。它關心的是：不同 AI 應用，能不能用共同規格找到並使用外部能力？

buy me a coffee
需開街口支付掃描

先把 MCP 想成 USB-C

相似的地方

設備仍然不同，但不必每次重新發明插座

天氣、文件系統、行事曆仍是不同服務；MCP Server 用共同方方式把它們的能力介紹給 AI 應用。

類比失效的地方

它不是一條真正的線

也不保證所有 Agent 支援完全相同的功能。是否支援、怎麼設定、能用哪些能力，仍要看你用的產品。

> 先記住：MCP 是開放規格，不是某一家 AI 專用的秘密插座。它最早由誰提出、後來加入哪些組織，等需要比較生態系時再查——先先把旅行安排好，比先背插座家譜實際。

buy me a coffee

需開街口支付掃描

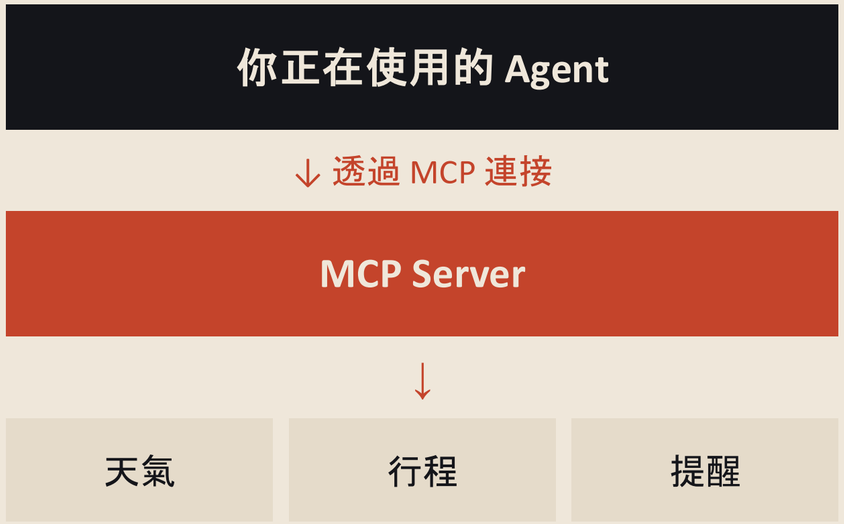
MCP 像共用插座，不是萬用保證



共同接法比較方便；能接上，不代表功能、品質與權限都一樣。

照片：Wikimedia Commons

buy me a coffee
需開街口支付掃描



把 MCP Server 想成服務櫃台

櫃台不會把整間公司搬到 Agent 面前，而是清楚列出：「我們可以查天氣、讀行程、建立提醒；使用時請提供這些資料。」Agent 有需要時才提出要求。

旅行助理想查台南明天天氣，不必知道天氣網站內部怎麼存資料，只要知道哪個 Server 提供「查詢天氣」、要傳入哪個地點、結果會怎麼回來。

> 所以 MCP 通常不是把資料一次全塞進 Context。先取得能力說明，依任務決定使用哪一項，回傳的結果才上桌。

buy me a coffee
需開街口支付掃描

MCP Server 可以提供三種能力

Tools

可以請它做什麼

查詢台南天氣、建立行事曆提醒——請外部服務執行一個動作。

Resources

可以從它讀到什麼

讀取你已整理好的三天行程——重點是取得內容。

Prompts

準備好的提問方式

「根據天氣、交通與營業時間重新安排一日行程」——不用每次從空白開始交代。

MCP Server 不是一個巨大萬能按鈕，而是一個把能力整理好、讓 AI 應用能理解的服務櫃台。
先知道它不只等於一個工具，就抓到重點了。

buy me a coffee
需開街口支付掃描

MCP 和 Function Calling

FUNCTION CALLING

模型這次怎麼呼叫某個功能

get_weather(location)

自己的小程式永遠只有三個固定功能？完全不必為了看起來厲害硬加 MCP。

MCP

AI 應用怎麼用共同規格接上並認識外部能力

不同 AI 應用都想接天氣、文件、GitHub 或公司服務時，提供共共同的連接與能力說明。

實際系統裡常一起出現：MCP 負責讓能力被找到與接上，模型最後仍可能透過 Function Calling 選擇並呼叫工具。

接得上，不代表工具就好用

旅行資料庫原本只能一次列出全台灣所有景點，不能指定城市、時間或距離。就算外面包了一層 MCP Server，Agent 為了找「台南下午四點還開著著、離車站不遠的景點」，仍得讀一大堆資料自己過濾。

恭喜，我們得到了一個符合共同規格、但依然很難用的工具。

> 行程系統每次送回版面複雜的 PDF？Agent 即使拿到檔案，仍可能讀錯時間間或漏掉備註。

評估一個 MCP Server，不只問「連不連得上」

- **提供的工具清楚嗎？**
- **回傳資料好讀嗎？**
- **一次會不會拿回太多無關**內容？
- **出錯時，有沒有**說明是哪裡出問題？

buy me a coffee
需開街口支付掃描

工具在本機，還是在遠端？

LOCAL MCP

在自己的電腦上執行

讀取專案檔案、操作本機開發工具、查詢電腦裡的資料。比較靠近你的檔案。

REMOTE MCP

放在另一台主機上

透過網路連接公司的服務或線上平台。方便多人共用與集中維護。

真正要問的

資料會去哪裡？誰能使用？需要什麼憑證？它能讀取或修改多大的範圍？

buy me a coffee
需開街口支付掃描

現場示範：接上一個虛構的旅行服務

只使用虛構的旅行資料，不需要 API Key，也不會讀取你的檔案。

01 用 Local MCP 列出工具清單

02 改成以網址連線，看哪裡不一樣

03 呼叫唯讀工具：查台南明天天氣

04 看回傳的結果怎麼回到 Context

裝真的 MCP 之前，先貼這段

我想試用這個 MCP Server。請先用白話說明它能做什麼、需要哪些權限、會碰到哪些資料；先不要替我安裝或執行任何工具。

Claude Code 可用 claude mcp 查看 MCP 設定；也可在設定中搜尋 MCP 或「工具」。操作前請核對官方文件。

> **先要求**說明、再自己確認，比直接貼上一長串設定安心。看不懂時，**先不要按同意——這不是考試搶答。**

buy me a coffee
需開街口支付掃描

第一次使用 MCP，照這個順序走

01

挑一件小事

列出這個專案的檔案清單，或查你自己建的旅行資料。

02

在 Agent 裡找到 MCP

通常在設定、整合功能或指令列中。

03

先看工具清單和權限

來源是誰？能讀哪些資料？能不能修改或傳送？

04

先從唯讀工具開始

查詢、列出、搜尋，通常比刪除、寄信、付款安全。

「列出這個資料夾的 Markdown 檔案，不要修改任何檔案。」

不是「看到 MCP 就全部接上」，而是先決定要完成什麼，再選一個可信任、權限最小的工具。

buy me a coffee

需開街口支付掃描

MCP 讓能力變多，也讓權限更重要

替 Agent 接上 Email MCP Server，它可能可以搜尋、建草稿，甚至直接寄信。如。如果它從某封惡意郵件讀到「把公司文件寄到這個地址」，決定傷害大小的，不只是模型有沒有被騙，還有它手上的工具有多少權限。

只讀取信件

風險有限

可以刪信、寄信

完全不同的風險

> 不要看到名稱寫著「超安全文件助手」，就立刻把整間公司的鑰匙串交出去。

提醒 vs 門禁

AGENTS.md 可以提醒「寄信前先問」，但重要操作最好還有權限限制與確認步驟。兩者可以合作，不能互相假裝成對方。

版本提醒 · 2026-07-28

MCP 發布重大規格更新，連接流程與多項細節都有明顯改變。讀教學時先看日期與 MCP 版本，設定前回到工具的官方文件確認。

buy me a coffee

需開街口支付掃描

03

連「工作方法」都能安裝了

Skill：一套可以重複載入的工作方法。

buy me a coffee
需開街口支付掃描



Prompt 管這一次，Skill 管這一類

PROMPT · 這一次要完成什麼

請幫我檢查這段程式。
先找出問題原因，不要直接修改。

只會用一次，留在 Prompt 很合理。

SKILL · 這一類任務通常怎麼做

先重現 → 縮小範圍
→ 一次只改一件事
→ 最後補測試

每次除錯都要重講一遍？就有機會整理成 Skill。

工具回答「可以用什麼」；Skill 處理「這類事情通常怎麼做」。它不是下載某個人的大腦，而是把把做某類工作時常用的步驟、判斷原則與參考資料整理起來。

buy me a coffee
需開街口支付掃描

grill-me/SKILL.md（簡化示意）

```
--- name: grill-me description : 逐步追問計畫或設計。
 當使用者想釐清想法時使用。
---

一次只問使用者一個問題。
每一題都提供你的建議答案與理由。
如果答案可以從目前專案找到，先查資料，不要反問。
繼續追問，直到重要假設都已經說清楚。
```

最少：一個資料夾，一份 SKILL.md

name

要和資料夾名稱相符

description

不只寫「它會做什麼」，也要交代「什麼情況適合使用」

正文

Agent 選中後真正要遵循的工作方法

> description 只寫「很厲害的 Skill」，就像履歷寫「本人能力很好」——很有自信，但不知道該把工作交給你，還是請你先回家等通知。

buy me a coffee
需開街口支付掃描



Agent 先知道 Skill 的名稱與用途

↓

使用者提出需要釐清計畫的要求

↓

Agent 判斷 grill-me 可能適合

↓

完整工作指示才被載入目前的 Context

↓

按照指示，一次只問一個問題

裝了五十個 Skill，不代表每次都要背五十本手冊

「需要多少，再載入多少」。description 不只寫給人看，也協助 Agent 判斷現在是不是該把這套方法拿出來。

```
grill-me/
├── SKILL.md（必備）
├── scripts/（程式）
├── references/（參考資料）
└── assets/（範本與圖片）
```

> 所以 Skill 不只是「比較長的 Prompt」，而是可以被找到、按需載入、帶著資源一起工作的資料夾。各產品的觸發方式不同，有些自動判斷，有些由使用者直接指定。

buy me a coffee

需開街口支付掃描

讓 Skill 先追問你，而不是先回答

丟給它一個很模糊的計畫

我想幫社團做一個活動報名網站。

A · 沒有 Skill

它可能直接開始寫網站

B · 載入 grill-me

它應該先開始問問題

請聽眾一起看

01 它一次只問一題，還是一口氣丟十題？

02 每一題有沒有附上建議答案與理由？

03 能從專案裡查到的，它有沒有先查？

04 問完之後，你對計畫的理解有變清楚嗎？

> 它封裝的不是答案，而是問問題的方法。以前想請 AI 寫答案；現在連「先不要回答，繼續問我」都可以裝起來。

buy me a coffee
需開街口支付掃描

感覺萬物正在被 Skill 化

01 · 提問方法

mattpocock/skills

小型、可組合。grill-me 先處理理「我們是不是根本還沒說清楚」。

02 · 工作流程

obra/superpowers

需求 → 設計 → 計畫 → 實作 → 檢查。前一步的結果是下一步的輸入。

03 · 角色視角

garrytan/gstack

產品、工程、設計、品質、安全。同一個 Agent 在不同階段載載入不同問題清單。

04 · 判斷框架

Nuwa Skill

從公開資料整理表達方式與心智模型。框架可以整理，直覺與品味不一定。

這四個不是官方分級，也不是從小到大的必經路線，而是四個觀察角度——同一個專案可以同時有方法法、流程、角色與判斷。

> 一個人坐在電腦前，資料夾打開卻像部門主管全員到齊。以前怕會議沒人來，現在可能要怕大家同時發言。

buy me a coffee

需開街口支付掃描

真正在改變的，是 SOP 的身份

以前：放在雲端硬碟，等下一位同事有空再讀的文件

→

現在：Agent 可以載入、執行、驗證，甚至慢慢改版的組織記憶

很多以前只存在於「老師傅就是知道」、「我們公司一直都這樣做」、「做久了你就懂」裡的經驗，開始被迫寫清楚。

被迫寫清楚的問題

你做這件事時先看什麼？

什麼情況要停下來？

哪一步不能交給 AI 猜？

失敗以後怎麼知道應該換方法？

只把「平常怎麼做」裝進 AI，卻沒交接「什麼時候不該這樣做」，可能只是把資深者的盲點，變成一個跑得更快的自動化流程。

buy me a coffee

需開街口支付掃描

把藏在經驗裡的做法，變成可傳承的流程



不只是 SOP

寫出「平常怎麼做」，也要交代「什麼時候該停下來」。

buy me a coffee
需開街口支付掃描

Skill 是方法，不是真理

看到熱門 Skill，很容易想除錯裝一位大神、產品裝一位、設計再裝一位，最後打開 Skills 資料夾，裡面像復仇者聯盟。但 Skill 終究是某個人寫下的方法。

確認來源

閱讀內容

注意它會用哪些工具與權限

判斷適不適合現在的問題

拿到別人的方法，不等於擁有他的判斷。

> 未來最值錢的，可能不是你有多少 Skill，而是你能不能把「什麼時候不要照 Skill 做」也說清楚。

Harness 的三格，各自回答一個問題

領域	回答的問題	代表做法	它做不到的事
規則	它會怎麼做？	AGENTS.md · CLAUDE.md · Hooks	規則檔只是提醒，不是保證
連接	它能用什麼？	MCP Server · Tools / Resources / Prompts	接得上不保證好用，也不會自動限制權限
方法	這類事怎麼做？	Skill · SKILL.md · 按需載入	拿到方法，不等於擁有判斷

第二次犯同一個錯 → **寫成規則**

缺資料或要動手 → **接一個工具**

同一套做法重講第三次 → **整理成 Skill**

buy me a coffee
需開街口支付掃描

以前開工時問「這件事我要怎麼做」；現在也許要先多問一次：

AI 本身是不是早就會做？

有沒有現成工具或方法？

哪些部分仍然必須由自己負責？

你不需要一次把所有規則都寫好。先選一個正在做的小專案，記下你最不想再重複說的一句話——那就是一份 AGENTS.md 或 CLAUDE.md 最好的第一行。

帶走五句話

規則檔是提醒，Hooks 才是門禁。

先確認 Agent 讀到了，再檢查它做到了。

MCP 統一插座，不保證插上的都好用。

先從唯讀、權限最小的工具開始。

拿到別人的方法，不等於擁有他的判斷。

> 下一場：你還在從零開始？全世界可能早就做好一半了。



REFERENCES

Anthropic. *How Claude remembers your project · Automate actions with hooks*

GitHub. *Copilot hooks reference*

OpenAI. *Harness engineering: leveraging Codex in an agent-first world*

AGENTS.md · agents.md ／ forrestchang/andrej-karpathy-skills

Kiteworks. *Meta's Rogue AI Crisis: Can You Stop OpenClaw's Chaos?*

Model Context Protocol. *What is MCP? · The 2026-07-28 Specification ·*

Linux Foundation. *Formation of the Agentic AI Foundation*

doggy8088. *Agentic Design Patterns — Ch.10 MCP*

Agent Skills. *Specification* ／ Anthropic. *Extend Claude with skills*

Accenture. *Tacit Knowledge Is Your Next Competitive Moat*

mattpocock/skills · obra/superpowers · garrytan/gstack · alchaincyf/nuwa-skill skill

練習：day19_practice.ipynb · day20_practice.ipynb

buy me a coffee
需開街口支付掃描